MYSQL & THIẾT KẾ CƠ SỞ DỮ LIỆU · CHƯƠNG TỔNG HỢP: GIÁO TRÌNH VÀ THỰC HÀNH · BÀI TOÀN KHÓA

MySQL & Thiết Kế Cơ Sở Dữ Liệu

Tài liệu gốc kèm thực hành bổ sung · 29/10/2026

Mục lục

- Nhập Môn Cơ Sở Dữ Liệu: Khái niệm cơ sở dữ liệu
- Nhập Môn Cơ Sở Dữ Liệu: Cài đặt MySQL & Workbench
- Nhập Môn Cơ Sở Dữ Liệu: Câu lệnh SELECT cơ bản
- Truy Vấn SQL Nâng Cao: JOIN nhiều bảng
- Truy Vấn SQL Nâng Cao: GROUP BY & Aggregate Function
- Truy Vấn SQL Nâng Cao: Subquery & View
- Thiết Kế Database Chuẩn Hoá: Chuẩn hoá dữ liệu (1NF-3NF)
- Thiết Kế Database Chuẩn Hoá: Thiết kế ERD
- Thiết Kế Database Chuẩn Hoá: Dự án: Thiết kế CSDL website học trực tuyến

CHƯƠNG 1 · BÀI 1

1.1. Khái niệm cơ sở dữ liệu

Tài liệu học tập MySQL & Thiết Kế Cơ Sở Dữ Liệu · Nội dung biên soạn trong dự án StudyOnline.

Mục tiêu bài học

- Phân biệt cơ sở dữ liệu và hệ quản trị cơ sở dữ liệu.
- Mô tả bảng, hàng, cột, khóa chính và khóa ngoại.
- Nhận biết quan hệ một-một, một-nhiều và nhiều-nhiều.
- Phân biệt các nhóm câu lệnh SQL trong tài liệu.

1. Cơ sở dữ liệu là gì

Cơ sở dữ liệu (CSDL, database) là tập hợp dữ liệu có tổ chức, được lưu trữ có cấu trúc để dễ dàng truy xuất, cập nhật và quản

lý. Hệ quản trị cơ sở dữ liệu (DBMS — Database Management System) là phần mềm trung gian cho phép người dùng/ứng

dụng tương tác với dữ liệu mà không cần thao tác trực tiếp lên file lưu trữ vật lý.

2. CSDL quan hệ (Relational Database)

CSDL quan hệ tổ chức dữ liệu thành các bảng (table) gồm hàng (row/record) và cột (column/field), các bảng có thể liên kết

với nhau thông qua khóa (key). MySQL, PostgreSQL, SQL Server, Oracle là các hệ quản trị CSDL quan hệ phổ biến, đều dùng

ngôn ngữ SQL (Structured Query Language) để thao tác dữ liệu.

3. Khóa chính và khóa ngoại

Khóa chính (Primary Key) là một hoặc nhiều cột có giá trị duy nhất, không được trùng và không được rỗng (NULL), dùng để

định danh chính xác từng hàng trong bảng — ví dụ cột id tự động tăng. Khóa ngoại (Foreign Key) là cột trong một bảng

tham chiếu tới khóa chính của bảng khác, dùng để thiết lập mối quan hệ giữa các bảng, đảm bảo tính toàn vẹn dữ liệu (ví dụ

không thể thêm một đơn hàng với mã khách hàng không tồn tại).

4. Các loại quan hệ giữa bảng

Quan hệ một-một (1-1): mỗi hàng ở bảng A tương ứng đúng một hàng ở bảng B, ví dụ bảng Người dùng và bảng Hồ sơ chi

tiết. Quan hệ một-nhiều (1-N): một hàng ở bảng A liên kết với nhiều hàng ở bảng B, ví dụ một Khách hàng có nhiều Đơn

hàng — đây là loại quan hệ phổ biến nhất. Quan hệ nhiều-nhiều (N-N): cần một bảng trung gian (junction table) chứa hai

khóa ngoại trỏ tới cả hai bảng liên quan, ví dụ Học viên và Khóa học (một học viên học nhiều khóa, một khóa có nhiều học

viên) cần thêm bảng Ghi_danh ở giữa.

5. SQL và phân loại câu lệnh

SQL chia thành các nhóm câu lệnh: DDL (Data Definition Language — CREATE, ALTER, DROP: định nghĩa/thay đổi cấu trúc

bảng), DML (Data Manipulation Language — SELECT, INSERT, UPDATE, DELETE: thao tác dữ liệu), DCL (Data Control Language

— GRANT, REVOKE: phân quyền truy cập), TCL (Transaction Control Language — COMMIT, ROLLBACK: quản lý giao dịch).

6. Mở rộng: NoSQL và khi nào không chọn CSDL quan hệ

Ngoài CSDL quan hệ, còn có CSDL NoSQL (MongoDB, Redis, Cassandra) không tổ chức dữ liệu theo bảng cố định, phù hợp

khi dữ liệu có cấu trúc linh hoạt thay đổi liên tục, cần mở rộng theo chiều ngang (horizontal scaling) ở quy mô cực lớn, hoặc

chỉ cần lưu trữ khóa-giá trị đơn giản tốc độ cao. Với hầu hết ứng dụng web/quản lý thông thường (như hệ thống LMS học

trực tuyến), CSDL quan hệ vẫn là lựa chọn phù hợp và phổ biến nhất nhờ tính nhất quán dữ liệu chặt chẽ (ACID).

Câu hỏi ôn tập và bài tập vận dụng

1. Phân biệt CSDL và DBMS. MySQL thuộc loại nào?
2. Vì sao khóa chính phải duy nhất và không NULL?
3. Thiết kế ba bảng học viên, khóa học và ghi danh: xác định khóa chính, khóa ngoại và loại quan hệ.

4. Phân loại CREATE, INSERT, GRANT và COMMIT theo chức năng.
5. Nêu một trường hợp cần cân nhắc lưu trữ khóa-giá trị thay cho bảng quan hệ.

Khái niệm cơ sở dữ liệu — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Hỏi thực tế: "Hai học sinh có cùng id được không?" Nếu id là khóa chính trong cùng bảng thì không: khóa chính định danh hàng, phải duy nhất và không NULL. Hai bảng khác nhau có thể có cùng giá trị số nhưng đó không tự là cùng thực thể.

Hỏi nối tiếp: "Bạn chưa xếp lớp thì mã lớp có để trống được không?" Đây là khóa ngoại, không phải khóa chính. Có thể để NULL nếu thiết kế cột cho phép và nghiệp vụ chấp nhận chưa xếp lớp. NOT NULL buộc có giá trị; giá trị khác NULL phải thỏa ràng buộc tham chiếu đang áp dụng. Nhiều học sinh được cùng mã lớp; đó là quan hệ một lớp–nhiều học sinh.

Tự luyện: vẽ bảng Lop và HocSinh. **Hướng giải:** HocSinh.id là khóa chính; HocSinh.lop_id tham chiếu Lop.id; không dùng tên làm khóa chính chỉ vì ví dụ chưa thấy trùng tên.

Nguồn học tiếp

https://dev.mysql.com/doc/refman/8.4/en/tutorial.html

CHƯƠNG 1 · BÀI 2

1.2. Cài đặt MySQL & Workbench

Tài liệu học tập MySQL & Thiết Kế Cơ Sở Dữ Liệu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Phân biệt MySQL Server và MySQL Workbench
> - Cài đặt MySQL và kết nối bằng Workbench
> - Tạo database, bảng đầu tiên và kiểm tra kết nối

1. Các thành phần

MySQL Server là thành phần lưu trữ và xử lý dữ liệu, mặc định lắng nghe ở cổng 3306. MySQL Workbench là công cụ giao diện đồ họa để viết truy vấn, quản lý bảng và vẽ sơ đồ ERD. Tài khoản quản trị mặc định của MySQL có tên là `root`, mật khẩu do bạn đặt lúc cài.

2. Các bước cài đặt

1. Tải MySQL Installer từ trang chủ MySQL và chọn MySQL Server cùng MySQL Workbench.
2. Đặt mật khẩu cho tài khoản `root` và ghi nhớ cổng (mặc định 3306).
3. Đảm bảo dịch vụ MySQL đang chạy (trên Windows xem trong Services).
4. Mở Workbench, tạo kết nối mới tới `127.0.0.1:3306`, kiểm tra bằng Test Connection.

Lưu ý: nếu máy đã có MySQL khác (như XAMPP) dùng cổng 3306, hãy đổi một trong hai sang cổng khác, ví dụ 3307, để tránh xung đột.

3. Tạo database và bảng đầu tiên

Bộ ký tự nên dùng là `utf8mb4` để lưu được tiếng Việt và emoji.

```
CREATE DATABASE hoc_truc_tuyen CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE hoc_truc_tuyen;

CREATE TABLE hoc_vien (
  id INT AUTO_INCREMENT PRIMARY KEY,
  ho_ten VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE,
  ngay_dang_ky DATE
```

```
);

SHOW TABLES;
DESCRIBE hoc_vien;
```

4. Kiểu dữ liệu thường dùng

Kiểu `INT` lưu số nguyên, `DECIMAL` lưu số thực chính xác thường dùng cho tiền tệ. Kiểu `VARCHAR` lưu chuỗi có độ dài thay đổi, `TEXT` lưu đoạn văn bản dài. Kiểu `DATE` lưu ngày, `DATETIME` và `TIMESTAMP` lưu cả ngày và giờ. Thuộc tính `AUTO_INCREMENT` tự tăng giá trị khoá chính khi thêm bản ghi mới.

Tóm tắt

- MySQL Server lưu và xử lý dữ liệu (cổng 3306); Workbench là công cụ giao diện.
- Dùng `utf8mb4` để lưu tiếng Việt.
- `CREATE DATABASE`, `CREATE TABLE`, `SHOW TABLES`, `DESCRIBE` là các lệnh nền tảng.

Câu hỏi ôn tập và bài tập vận dụng

1. Cài MySQL và Workbench, chụp lại kết nối thành công.
2. Tạo database và bảng `khoa_hoc` gồm id, tên, giá, ngày tạo.
3. Giải thích vì sao tiền tệ nên lưu bằng DECIMAL thay cho FLOAT.

Cài đặt MySQL & Workbench — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: Workbench mở được nhưng kết nối thất bại. Workbench là client; MySQL server phải chạy, đúng host/port, tài khoản và quyền. Phân biệt sai mật khẩu, server không chạy, firewall và database chưa tồn tại bằng thông báo lỗi.

```
SELECT VERSION();
SELECT DATABASE();
SHOW TABLES;
```

Tự luyện: kết nối lab, kiểm tra database hiện tại rồi mới chạy truy vấn. **Hướng giải:** không dùng tài khoản root trong ứng dụng chỉ vì dễ kết nối; cấp quyền tối thiểu phù hợp. Không import script có DROP DATABASE vào nơi cần giữ dữ liệu; sao lưu không thay thế việc xác nhận đúng database đích.

Nguồn học tiếp

https://dev.mysql.com/doc/refman/8.4/en/tutorial.html

CHƯƠNG 1 · BÀI 3

1.3. Câu lệnh SELECT cơ bản

Tài liệu học tập MySQL & Thiết Kế Cơ Sở Dữ Liệu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Viết câu lệnh SELECT truy vấn dữ liệu từ một bảng
> - Lọc bằng WHERE, sắp xếp bằng ORDER BY, giới hạn bằng LIMIT
> - Dùng các toán tử LIKE, IN, BETWEEN, IS NULL

1. Câu lệnh SELECT

Câu lệnh `SELECT` dùng để truy vấn, lấy dữ liệu từ một hoặc nhiều bảng. Ví dụ `SELECT name, price FROM products` lấy hai cột tên và giá của mọi sản phẩm. Dấu `*` trong `SELECT *` nghĩa là lấy tất cả các cột. Có thể đặt tên gọi khác cho cột bằng `AS`.

```
SELECT ho_ten, email FROM hoc_vien;
SELECT ho_ten AS ten, YEAR(ngay_dang_ky) AS nam FROM hoc_vien;
```

2. Lọc dữ liệu với WHERE

Mệnh đề `WHERE` dùng để lọc, chỉ giữ lại các bản ghi thoả điều kiện. Có thể kết hợp nhiều điều kiện bằng `AND`, `OR`, `NOT`.

```
SELECT * FROM khoa_hoc WHERE gia > 500000 AND trang_thai = 'published';
```

3. Các toán tử trong WHERE

Toán tử `LIKE` tìm theo mẫu, ký tự `%` đại diện cho chuỗi bất kỳ. Toán tử `IN` kiểm tra giá trị có nằm trong danh sách, `BETWEEN` kiểm tra giá trị nằm trong một khoảng. Để kiểm tra giá trị rỗng phải dùng `IS NULL`, không dùng dấu bằng.

```
SELECT * FROM hoc_vien WHERE ho_ten LIKE 'Nguyễn%';
SELECT * FROM khoa_hoc WHERE id IN (1, 3, 5);
SELECT * FROM khoa_hoc WHERE gia BETWEEN 100000 AND 500000;
SELECT * FROM hoc_vien WHERE email IS NULL;
```

4. Sắp xếp và giới hạn kết quả

Mệnh đề `ORDER BY` sắp xếp kết quả tăng dần (`ASC`) hoặc giảm dần (`DESC`). Từ khoá `DISTINCT` loại bỏ các dòng trùng lặp trong kết quả. Mệnh đề `LIMIT` giới hạn số dòng trả về, ví dụ `LIMIT 10` lấy 10 dòng đầu.

```
SELECT DISTINCT trinh_do FROM khoa_hoc;
SELECT * FROM khoa_hoc ORDER BY gia DESC LIMIT 5;
```

Tóm tắt

- `SELECT cột FROM bảng` lấy dữ liệu; `*` lấy mọi cột.
- `WHERE` lọc dòng; `LIKE`, `IN`, `BETWEEN`, `IS NULL` là các toán tử phổ biến.
- `ORDER BY` sắp xếp, `LIMIT` giới hạn, `DISTINCT` bỏ trùng.

Câu hỏi ôn tập và bài tập vận dụng

1. Lấy 5 khoá học có giá cao nhất.
2. Tìm các học viên có họ tên bắt đầu bằng chữ "Trần".
3. Giải thích vì sao phải dùng `IS NULL` thay vì `= NULL`.

Câu lệnh SELECT cơ bản — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: nhập tên O'Reilly làm hỏng SQL. Chuỗi có dấu nháy cần được truyền qua tham số của driver, không ghép chuỗi SQL bằng phép cộng hoặc f-string. Cú pháp placeholder tùy driver; không chép nhầm giữa các thư viện.

```
SELECT id, name
FROM students
WHERE name IS NOT NULL
ORDER BY name, id
LIMIT 20;
```

Tự luyện: lọc tên, sắp xếp ổn định và phân trang lab. **Hướng giải:** NULL được kiểm tra bằng IS NULL/IS NOT NULL, không bằng `= NULL`. Kết quả không có ORDER BY không được đảm bảo thứ tự; SELECT * tiện khám phá nhưng API nên chọn cột cần dùng và tránh lộ cột nhạy cảm.

Nguồn học tiếp

https://dev.mysql.com/doc/refman/8.4/en/tutorial.html

CHƯƠNG 2 · BÀI 1

2.1. JOIN nhiều bảng

Tài liệu học tập MySQL & Thiết Kế Cơ Sở Dữ Liệu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu phép JOIN và khi nào cần kết nối nhiều bảng
> - Phân biệt INNER JOIN, LEFT JOIN và RIGHT JOIN
> - Viết truy vấn kết nối từ ba bảng trở lên

1. JOIN là gì

JOIN là phép kết nối dùng để lấy dữ liệu từ nhiều bảng dựa trên cột liên quan, thường là khoá ngoại. Ví dụ kết nối bảng `lessons` với bảng `chapters` qua điều kiện `lessons.chapter_id = chapters.id`.

2. Phân biệt INNER JOIN và LEFT JOIN

`INNER JOIN` chỉ trả về các dòng có giá trị khớp ở cả hai bảng. `LEFT JOIN` trả về mọi dòng của bảng bên trái, các cột của bảng bên phải là `NULL` khi không có dòng khớp. `RIGHT JOIN` ngược lại với `LEFT JOIN`, giữ mọi dòng của bảng bên phải.

```
-- Khoá học kèm tên giảng viên
SELECT c.title, u.fullname
FROM courses c
INNER JOIN users u ON u.id = c.teacher_id;

-- Mọi khoá học, kể cả khoá chưa có học viên ghi danh
SELECT c.title, COUNT(e.id) AS so_hoc_vien
FROM courses c
LEFT JOIN enrollments e ON e.course_id = c.id
GROUP BY c.id, c.title;
```

3. Kết nối nhiều bảng

Có thể nối tiếp nhiều `JOIN` trong một truy vấn.

```
SELECT u.fullname, c.title, l.title AS bai_hoc
FROM users u
JOIN enrollments e ON e.user_id = u.id
JOIN courses c     ON c.id = e.course_id
JOIN chapters ch   ON ch.course_id = c.id
JOIN lessons l     ON l.chapter_id = ch.id;
```

4. Mẹo khi dùng JOIN

Luôn đặt bí danh cho bảng (`courses c`) và ghi rõ bí danh trước tên cột để tránh nhập nhằng. Điều kiện kết nối đặt ở mệnh đề `ON`. Dùng `LEFT JOIN` để tìm dữ liệu "không có": thêm điều kiện `WHERE bảng_phải.id IS NULL`, ví dụ khoá học chưa có học viên nào.

Tóm tắt

- JOIN kết nối bảng theo cột liên quan (thường là khoá ngoại).
- `INNER JOIN` chỉ giữ dòng khớp; `LEFT JOIN` giữ mọi dòng bảng trái.
- Đặt bí danh bảng và điều kiện `ON` rõ ràng.

Câu hỏi ôn tập và bài tập vận dụng

1. Liệt kê tên khoá học cùng tên giảng viên phụ trách.
2. Tìm các khoá học chưa có học viên nào (dùng LEFT JOIN và IS NULL).
3. Viết truy vấn liệt kê học viên, khoá học và số bài học trong khoá đó.

JOIN nhiều bảng — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Hỏi thực tế: "Tìm học viên chưa ghi danh khóa nào bằng LEFT JOIN?" Bắt đầu từ học viên, giữ tất cả học viên rồi lọc phía ghi danh không khớp. Đừng đổi thành tìm khóa chưa có học viên: đó là bài toán ngược.

```
SELECT s.id, s.name
FROM students AS s
LEFT JOIN enrollments AS e ON e.student_id = s.id
WHERE e.student_id IS NULL;
```

Tự luyện: tạo học viên A chưa ghi danh, B có hai ghi danh, C có một ghi danh. **Hướng giải:** truy vấn trên chỉ trả A; INNER JOIN không thể giữ A. LEFT JOIN giữ bảng trái, cột bên phải NULL khi không khớp. Khi lọc trạng thái ghi danh, điều kiện trong ON và trong WHERE có thể tạo kết quả khác nhau; kiểm tra bằng dữ liệu không có dòng khớp.

Nguồn học tiếp

https://dev.mysql.com/doc/refman/8.4/en/tutorial.html

CHƯƠNG 2 · BÀI 2

2.2. GROUP BY & Aggregate Function

Tài liệu học tập MySQL & Thiết Kế Cơ Sở Dữ Liệu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Sử dụng các hàm tổng hợp COUNT, SUM, AVG, MIN, MAX
> - Gom nhóm dữ liệu bằng GROUP BY
> - Phân biệt WHERE và HAVING

1. Hàm tổng hợp

Hàm tổng hợp (aggregate function) là hàm tính toán trên một nhóm dòng và trả về một giá trị duy nhất. Các hàm phổ biến gồm `COUNT` (đếm), `SUM` (tính tổng), `AVG` (trung bình), `MIN` (nhỏ nhất) và `MAX` (lớn nhất).

2. Hàm COUNT

Hàm `COUNT` dùng để đếm số dòng trong kết quả truy vấn. `COUNT(*)` đếm mọi dòng, còn `COUNT(tên_cột)` chỉ đếm các dòng có giá trị khác `NULL` ở cột đó. Ví dụ `SELECT COUNT(*) FROM hoc_vien` cho biết bảng có bao nhiêu học viên.

```
SELECT COUNT(*) AS tong, AVG(gia) AS gia_tb, MAX(gia) AS gia_cao_nhat
FROM khoa_hoc;
```

3. GROUP BY

Mệnh đề `GROUP BY` dùng để gom các dòng có cùng giá trị thành nhóm, thường đi cùng hàm tổng hợp. Ví dụ đếm số học viên của từng khoá học:

```
SELECT course_id, COUNT(*) AS so_hoc_vien
FROM enrollments
GROUP BY course_id;
```

4. Phân biệt WHERE và HAVING

Điểm khác nhau giữa WHERE và HAVING là thời điểm lọc dữ liệu. `WHERE` lọc từng dòng trước khi gom nhóm và không dùng được hàm tổng hợp. `HAVING` lọc các nhóm sau khi đã `GROUP BY` và dùng

được hàm tổng hợp, ví dụ `HAVING COUNT(*) > 5`.

```
SELECT course_id, COUNT(*) AS so_hoc_vien
FROM enrollments
WHERE enroll_date >= '2026-01-01'
GROUP BY course_id
HAVING COUNT(*) >= 5
ORDER BY so_hoc_vien DESC;
```

Thứ tự viết các mệnh đề: `SELECT`, `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`.

Tóm tắt

- `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` tính toán trên nhóm dòng; `COUNT(*)` đếm cả dòng NULL.
- `GROUP BY` gom nhóm; `HAVING` lọc nhóm, `WHERE` lọc dòng.
- Giữ đúng thứ tự các mệnh đề khi viết truy vấn.

Câu hỏi ôn tập và bài tập vận dụng

1. Đếm số bài học của từng khoá học.
2. Tính điểm trung bình quiz của từng học viên và chỉ giữ người có trung bình từ 5 trở lên.
3. Giải thích khác biệt giữa `WHERE` và `HAVING` kèm ví dụ.

GROUP BY & Aggregate Function — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: đếm học viên theo khóa nhưng khóa rỗng vẫn có số 1. Sau LEFT JOIN, COUNT(*) đếm cả hàng bên trái chưa khớp; COUNT(e.id) chỉ đếm id phía ghi danh không NULL.

```
SELECT c.id, c.title, COUNT(e.id) AS student_count
FROM courses AS c
LEFT JOIN enrollments AS e ON e.course_id = c.id
GROUP BY c.id, c.title;
```

Tự luyện: khóa có 0, 1, 3 ghi danh và dữ liệu bị nhân dòng bởi JOIN khác. **Hướng giải:** đếm ở đúng mức dữ liệu hoặc dùng subquery tổng hợp trước; không thêm DISTINCT vô điều kiện chỉ để che JOIN sai. WHERE lọc hàng trước nhóm; HAVING lọc kết quả nhóm. AVG bỏ NULL nên phải nêu rõ cách xử lý người chưa có điểm.

Nguồn học tiếp

https://dev.mysql.com/doc/refman/8.4/en/tutorial.html

CHƯƠNG 2 · BÀI 3

2.3. Subquery & View

Tài liệu học tập MySQL & Thiết Kế Cơ Sở Dữ Liệu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Viết truy vấn con (subquery) trong WHERE, FROM và SELECT
> - Tạo và sử dụng View
> - Hiểu khi nào nên dùng subquery, JOIN hay View

1. Truy vấn con

Subquery (truy vấn con) là câu truy vấn đặt bên trong một câu truy vấn khác. Truy vấn con có thể nằm trong `WHERE`, `FROM` hoặc `SELECT`. Ví dụ tìm sản phẩm có giá cao hơn giá trung bình:

```
SELECT title, price
FROM courses
WHERE price > (SELECT AVG(price) FROM courses);
```

2. Truy vấn con với IN và EXISTS

`IN` so sánh với danh sách giá trị do truy vấn con trả về; `EXISTS` kiểm tra truy vấn con có trả về dòng nào không, thường nhanh hơn khi chỉ cần biết có hay không.

```
-- Học viên đã ghi danh ít nhất một khoá
SELECT fullname FROM users u
WHERE EXISTS (SELECT 1 FROM enrollments e WHERE e.user_id = u.id);

-- Khoá học có ít nhất một quiz
SELECT title FROM courses
WHERE id IN (SELECT course_id FROM quizzes);
```

3. View

View là bảng ảo được tạo từ kết quả của một câu truy vấn, khai báo bằng `CREATE VIEW`. View không lưu dữ liệu riêng mà lấy dữ liệu từ các bảng gốc mỗi khi được truy vấn. View giúp đơn giản hoá truy vấn phức tạp và giới hạn cột mà người dùng được xem.

```
CREATE VIEW v_thong_ke_khoa_hoc AS
SELECT c.id, c.title, COUNT(e.id) AS so_hoc_vien
FROM courses c
LEFT JOIN enrollments e ON e.course_id = c.id
GROUP BY c.id, c.title;

SELECT * FROM v_thong_ke_khoa_hoc WHERE so_hoc_vien > 0;
DROP VIEW v_thong_ke_khoa_hoc;
```

4. Chọn cách viết phù hợp

Dùng JOIN khi cần lấy cột từ nhiều bảng; dùng subquery khi cần so sánh với kết quả tổng hợp hoặc kiểm tra tồn tại; dùng View khi một truy vấn được dùng lại nhiều lần hoặc cần ẩn bớt dữ liệu nhạy cảm. Với dữ liệu lớn nên kiểm tra kế hoạch thực thi bằng `EXPLAIN`.

Tóm tắt

- Subquery là truy vấn lồng trong `WHERE`, `FROM` hoặc `SELECT`.
- `IN` so với danh sách; `EXISTS` kiểm tra tồn tại.
- View là bảng ảo không lưu dữ liệu, giúp đơn giản hoá và hạn chế quyền xem.

Câu hỏi ôn tập và bài tập vận dụng

1. Tìm các khoá học có giá cao hơn giá trung bình bằng subquery.
2. Tạo view liệt kê học viên kèm số khoá học đã ghi danh.
3. Giải thích View có lưu dữ liệu riêng hay không.

Subquery & View — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: cần tìm học viên không có ghi danh mà không muốn nhân dòng. NOT EXISTS thể hiện trực tiếp điều kiện không tồn tại.

```
SELECT s.id, s.name
FROM students AS s
WHERE NOT EXISTS (
    SELECT 1 FROM enrollments AS e WHERE e.student_id = s.id
);
```

Tự luyện: đối chiếu kết quả với bài LEFT JOIN. **Hướng giải:** hai cách đúng phải cho cùng tập học viên trên dữ liệu lab. Cẩn thận NOT IN khi subquery có NULL; View là truy vấn có tên, không mặc nhiên là bảng sao chép được cập nhật sẵn hoặc biện pháp tăng tốc mọi truy vấn.

Nguồn học tiếp

https://dev.mysql.com/doc/refman/8.4/en/tutorial.html

CHƯƠNG 3 · BÀI 1

3.1. Chuẩn hoá dữ liệu (1NF-3NF)

Tài liệu học tập MySQL & Thiết Kế Cơ Sở Dữ Liệu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu vì sao cần chuẩn hoá dữ liệu
> - Phân biệt các dạng chuẩn 1NF, 2NF và 3NF
> - Áp dụng chuẩn hoá để tách một bảng thành nhiều bảng

1. Chuẩn hoá là gì

Chuẩn hoá (normalization) là quá trình tổ chức bảng để giảm dư thừa dữ liệu và tránh lỗi cập nhật. Dư thừa dữ liệu gây ra các dị thường (anomaly) khi thêm, sửa và xoá: ví dụ lưu tên khoá học lặp lại ở hàng nghìn dòng thì sửa tên phải sửa từng dòng. Các dạng chuẩn thường dùng là 1NF, 2NF và 3NF.

2. Dạng chuẩn 1NF

Bảng đạt dạng chuẩn 1NF khi mọi cột chỉ chứa giá trị nguyên tố, không có nhóm lặp. Ví dụ cột số điện thoại chứa nhiều số cách nhau bởi dấu phẩy là vi phạm 1NF; cần tách thành các dòng hoặc bảng riêng.

3. Dạng chuẩn 2NF

Bảng đạt dạng chuẩn 2NF khi đạt 1NF và mọi cột không khoá phụ thuộc đầy đủ vào toàn bộ khoá chính. 2NF chỉ cần xét khi khoá chính gồm nhiều cột, loại bỏ phụ thuộc vào một phần của khoá.

Ví dụ bảng `dang_ky(ma_hv, ma_khoa, ten_khoa, ngay)` có khoá chính (ma_hv, ma_khoa): cột `ten_khoa` chỉ phụ thuộc `ma_khoa` nên vi phạm 2NF. Tách `ten_khoa` ra bảng `khoa_hoc`.

4. Dạng chuẩn 3NF

Bảng đạt dạng chuẩn 3NF khi đạt 2NF và không có phụ thuộc bắc cầu giữa các cột không khoá. Phụ thuộc bắc cầu nghĩa là cột C phụ thuộc cột B, còn cột B lại phụ thuộc khoá chính A.

Ví dụ bảng `hoc_vien(ma_hv, ten, ma_lop, ten_lop)`: `ten_lop` phụ thuộc `ma_lop`, còn `ma_lop` phụ thuộc `ma_hv` nên có phụ thuộc bắc cầu. Tách bảng `lop(ma_lop, ten_lop)`.

```
CREATE TABLE lop (ma_lop INT PRIMARY KEY, ten_lop VARCHAR(100));
CREATE TABLE hoc_vien (
  ma_hv INT PRIMARY KEY, ten VARCHAR(100),
  ma_lop INT, FOREIGN KEY (ma_lop) REFERENCES lop(ma_lop)
);
```

Lưu ý

Chuẩn hoá quá mức có thể làm truy vấn phải JOIN nhiều bảng. Trong thực tế đôi khi cố ý phi chuẩn hoá một phần để tăng tốc độ đọc, nhưng cần hiểu rõ đánh đổi.

Tóm tắt

- Chuẩn hoá giảm dư thừa và tránh dị thường khi thêm, sửa, xoá.
- 1NF: giá trị nguyên tố; 2NF: không phụ thuộc một phần vào khoá; 3NF: không phụ thuộc bắc cầu.
- Tách bảng và nối lại bằng khoá ngoại.

Câu hỏi ôn tập và bài tập vận dụng

1. Cho bảng đơn hàng lưu nhiều sản phẩm trong một ô; đưa về 1NF.
2. Chỉ ra phụ thuộc bắc cầu trong bảng `nhan_vien(ma_nv, ten, ma_pb, ten_pb)` và tách bảng.
3. Nêu một trường hợp nên phi chuẩn hoá và lý do.

Chuẩn hoá dữ liệu (1NF-3NF) — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: lưu nhiều tên khóa trong một ô khiến sửa/tìm rất khó. Tách thực thể và quan hệ theo phụ thuộc dữ liệu: học viên, khóa, ghi danh. Khóa ghép của ghi danh hoặc unique(student_id, course_id) ngăn đăng ký trùng theo quy tắc nghiệp vụ.

Tự luyện: từ bảng gồm student_id, student_name, course_id, course_title và teacher_name, chỉ ra việc sửa tên khóa phải sửa nhiều dòng. **Hướng giải:** đưa tên khóa vào bảng courses, nối bằng course_id; tiếp tục xem giảng viên có định danh riêng không. Chuẩn hóa dựa phụ thuộc hàm, không chỉ dựa cảm giác “tách nhiều bảng là tốt”. Đo hiệu năng trước khi phi chuẩn hóa có chủ đích.

Nguồn học tiếp

https://dev.mysql.com/doc/refman/8.4/en/tutorial.html

CHƯƠNG 3 · BÀI 2

3.2. Thiết kế ERD

Tài liệu học tập MySQL & Thiết Kế Cơ Sở Dữ Liệu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu ERD và các thành phần thực thể, thuộc tính, liên kết
> - Xác định các kiểu liên kết 1-1, 1-N, N-N
> - Chuyển ERD thành các bảng trong MySQL

1. Sơ đồ ERD

ERD (Entity Relationship Diagram) là sơ đồ thực thể - liên kết dùng để mô hình hoá cơ sở dữ liệu trước khi tạo bảng. Thực thể (entity) là đối tượng cần lưu trữ, ví dụ học viên hoặc khoá học. Thuộc tính là đặc điểm của thực thể (họ tên, email). Liên kết thể hiện mối quan hệ giữa các thực thể.

2. Các kiểu liên kết

Các kiểu liên kết gồm một - một (1-1), một - nhiều (1-N) và nhiều - nhiều (N-N).

- **1-1**: mỗi người dùng có một hồ sơ.
- **1-N**: một giảng viên phụ trách nhiều khoá học, mỗi khoá thuộc một giảng viên.
- **N-N**: một học viên học nhiều khoá và một khoá có nhiều học viên.

3. Chuyển ERD sang bảng

- Liên kết 1-N: đặt khoá ngoại ở phía "nhiều" (`courses.teacher_id` tham chiếu `users.id`).
- Liên kết N-N: tách thành một bảng trung gian chứa hai khoá ngoại, ví dụ bảng `enrollments` gồm `user_id` và `course_id`.
- Liên kết 1-1: đặt khoá ngoại có ràng buộc `UNIQUE`, hoặc gộp hai bảng.

```
CREATE TABLE enrollments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  course_id INT NOT NULL,
  enroll_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (user_id, course_id),
  FOREIGN KEY (user_id) REFERENCES users(id),
```

```
  FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

4. Quy trình thiết kế

Liệt kê thực thể từ yêu cầu bài toán, xác định thuộc tính và khoá chính, xác định liên kết giữa các thực thể, vẽ ERD (có thể dùng Workbench hoặc draw.io), kiểm tra chuẩn hoá đến 3NF rồi mới tạo bảng.

Tóm tắt

- ERD gồm thực thể, thuộc tính và liên kết.
- 1-N đặt khoá ngoại phía nhiều; N-N cần bảng trung gian.
- Thiết kế ERD và chuẩn hoá trước khi tạo bảng.

Câu hỏi ôn tập và bài tập vận dụng

1. Vẽ ERD cho hệ thống thư viện gồm sách, độc giả và phiếu mượn.
2. Chuyển ERD đó thành các câu lệnh CREATE TABLE có khoá ngoại.
3. Giải thích vì sao liên kết nhiều - nhiều cần bảng trung gian.

Thiết kế ERD — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: chưa rõ quan hệ học viên–khóa là 1-N hay N-N. Một học viên học nhiều khóa, một khóa có nhiều học viên nên cần bảng ghi danh ở giữa. Ghi cả tính bắt buộc/tùy chọn và quy tắc xóa.

Tự luyện: xác định khóa chính, khóa ngoại và unique cho users, courses, enrollments. **Hướng giải:** mô tả điều gì xảy ra khi khóa đã có điểm bị xóa; không mặc định CASCADE phù hợp mọi dữ liệu lịch sử. Chỉ số hoặc khóa duy nhất hỗ trợ tính toàn vẹn, còn quyền ai được đọc/sửa vẫn phải được ứng dụng kiểm soát.

Nguồn học tiếp

https://dev.mysql.com/doc/refman/8.4/en/tutorial.html

CHƯƠNG 3 · BÀI 3

3.3. Dự án: Thiết kế CSDL website học trực tuyến

Tài liệu học tập MySQL & Thiết Kế Cơ Sở Dữ Liệu · Nội dung biên soạn trong dự án StudyOnline.

Mục tiêu bài học

- Thiết kế cơ sở dữ liệu hoàn chỉnh cho website học trực tuyến
- Áp dụng ERD, chuẩn hoá và khoá ngoại vào một hệ thống thực tế
- Viết một số truy vấn thống kê từ CSDL đã thiết kế

1. Yêu cầu hệ thống

Website cho phép giảng viên tạo khoá học gồm các chương và bài học, học viên ghi danh, học bài, làm quiz trắc nghiệm và theo dõi tiến độ; quản trị viên xem thống kê và thanh toán.

2. Các bảng chính

Các bảng chính gồm `users`, `courses`, `chapters`, `lessons`, `quizzes`, `questions` và `results`.

- `users(id, fullname, email, password, role)` với vai trò admin, teacher hoặc student.
- `courses(id, teacher_id, title, description, price, status)`.
- `chapters(id, course_id, chapter_name, order_index)` và `lessons(id, chapter_id, title, video_url, document_url, order_index)`.
- `quizzes(id, course_id, title)`, `questions(id, quiz_id, content, option_a..d, correct_answer)`.
- `results(id, user_id, quiz_id, score, total, submit_time)`.

3. Các liên kết

Bảng `enrollments` là bảng trung gian thể hiện quan hệ nhiều - nhiều giữa học viên và khoá học. Bảng `lessons` có khoá ngoại `chapter_id` tham chiếu tới bảng `chapters` theo quan hệ một - nhiều; `chapters` có khoá ngoại `course_id` tham chiếu `courses`. Bảng `lesson_progress(user_id, lesson_id, is_completed, watched_sec)` lưu tiến độ học của từng học viên trên từng bài.

4. Truy vấn thống kê mẫu

```
-- Số học viên của từng khoá học
SELECT c.title, COUNT(e.id) AS so_hoc_vien
```

```
FROM courses c LEFT JOIN enrollments e ON e.course_id = c.id
GROUP BY c.id, c.title
ORDER BY so_hoc_vien DESC;

-- Điểm trung bình quiz của từng học viên
SELECT u.fullname, ROUND(AVG(r.score * 100.0 / r.total), 1) AS diem_tb
FROM results r JOIN users u ON u.id = r.user_id
GROUP BY u.id, u.fullname;

-- Tỉ lệ hoàn thành bài học của học viên trong một khoá
SELECT COUNT(DISTINCT CASE WHEN lp.is_completed = 1 THEN l.id END) * 100.0 /
COUNT(DISTINCT l.id) AS ti_le
FROM chapters ch JOIN lessons l ON l.chapter_id = ch.id
LEFT JOIN lesson_progress lp ON lp.lesson_id = l.id AND lp.user_id = 6
WHERE ch.course_id = 1;
```

Tóm tắt

- Hệ thống học trực tuyến xoay quanh người dùng, khoá học, chương, bài, quiz và kết quả.
- Liên kết nhiều - nhiều giữa học viên và khoá học dùng bảng `enrollments`.
- Các báo cáo thống kê dựa trên JOIN, GROUP BY và hàm tổng hợp.

Câu hỏi ôn tập và bài tập vận dụng

1. Vẽ ERD đầy đủ cho hệ thống và tạo các bảng bằng MySQL.
2. Thêm bảng `payments` lưu giao dịch thanh toán khoá học và các khoá ngoại phù hợp.
3. Viết truy vấn tính doanh thu theo từng khoá học.

CHƯƠNG HỌC THÊM · BÀI TỔNG HỢP

Học thêm.Tổng hợp. Học thêm, bài tập và hướng dẫn giải

Tài liệu học tập MySQL & Thiết Kế Cơ Sở Dữ Liệu · Nội dung biên soạn trong dự án StudyOnline.

MySQL — Học thêm về truy vấn, ràng buộc và giao dịch

Tài liệu bổ sung cho SELECT, JOIN, GROUP BY, view, chuẩn hóa và dự án CSDL học trực tuyến. Ví dụ dành cho MySQL 8.4 và bảng InnoDB; thực hành trong database thử.

Mục tiêu học thêm

- Giải thích tác động của JOIN tới số dòng và thống kê
- Thiết kế ràng buộc chống dữ liệu trùng hoặc mất liên kết
- Sử dụng giao dịch cho thao tác có nhiều bước phụ thuộc

1. JOIN không chỉ ghép cột

Một khóa có nhiều học viên và nhiều bài học. Nếu JOIN cùng lúc hai quan hệ một-nhiều, số dòng có thể bị nhân lên. Với 3 học viên và 4 bài, kết quả có thể là 12 dòng chứ không phải 7.

```
SELECT c.id, COUNT(DISTINCT e.user_id) AS students,
       COUNT(DISTINCT l.id) AS lessons
FROM courses c
LEFT JOIN enrollments e ON e.course_id = c.id
LEFT JOIN chapters ch ON ch.course_id = c.id
LEFT JOIN lessons l ON l.chapter_id = ch.id
GROUP BY c.id;
```

DISTINCT phù hợp khi đếm mã duy nhất; không dùng SUM(DISTINCT amount) để chữa lỗi nhân dòng vì hai khoản tiền hợp lệ cùng giá trị sẽ bị gộp mất. Khi tính doanh thu cùng số bài, tổng hợp payments và lessons riêng trước rồi JOIN các kết quả tổng hợp.

2. NULL và điều kiện lọc

NULL biểu thị thiếu giá trị, không phải số 0 hay chuỗi rỗng. Dùng IS NULL, không dùng = NULL. COUNT(*) đếm dòng; COUNT(column) bỏ qua giá trị NULL.

Với LEFT JOIN, đưa điều kiện bảng phải xuống WHERE có thể loại các dòng không có bản ghi khớp, làm mất ý nghĩa giữ bảng trái.

```
SELECT c.id, COUNT(p.id) AS completed_payments
FROM courses c
LEFT JOIN payments p
  ON p.course_id = c.id AND p.status = 'completed'
GROUP BY c.id;
```

Khóa không có giao dịch thành công vẫn xuất hiện với số đếm 0. Nếu chuyển p.status='completed' xuống WHERE, khóa đó thường biến mất khỏi kết quả.

3. Khóa và chuẩn hóa

Một bản ghi ghi danh cần UNIQUE(user_id, course_id) để không ghi danh trùng. Kiểm tra trước bằng SELECT trong ứng dụng không đủ khi hai request chạy đồng thời; ràng buộc database là lớp bảo vệ cuối cùng.

Chuẩn hóa giảm dư thừa và lỗi cập nhật. Trong bảng ghi danh, tên khóa phụ thuộc course_id chứ không phụ thuộc toàn bộ cặp học viên/khóa; nên lưu tên tại courses. Tuy nhiên, lịch sử thanh toán có thể cần lưu giá mua tại thời điểm giao dịch để giá khóa thay đổi không làm sai lịch sử. Không xóa mọi dữ liệu lặp một cách máy móc: cần phân biệt dư thừa với snapshot nghiệp vụ.

4. Giao dịch và đồng thời

Khi xác nhận thanh toán, cập nhật trạng thái đơn và tạo quyền học cần được thực hiện nhất quán. Ví dụ dưới đây minh họa, cần điều chỉnh tên cột và xử lý ràng buộc theo schema thật:

```
START TRANSACTION;
SELECT id, user_id, course_id, status
FROM payments WHERE id = 123 FOR UPDATE;
-- Ứng dụng kiểm tra trạng thái, số tiền và tính hợp lệ trước khi tiếp tục.
UPDATE payments SET status = 'completed' WHERE id = 123 AND status = 'pending';
-- Ứng dụng chỉ thêm ghi danh khi đủ điều kiện nghiệp vụ.
COMMIT;
```

Không chạy nguyên ví dụ trên để xác nhận giao dịch thật. Luồng đầy đủ phải thêm ghi danh trong cùng giao dịch, xử lý lỗi bằng rollback và bảo đảm callback lặp không tạo quyền học trùng. FOR UPDATE cần giao dịch đang mở và engine hỗ trợ khóa hàng như InnoDB. Giao dịch không thay thế việc kiểm tra chữ ký thanh toán.

5. Chỉ mục và kế hoạch thực thi

Chỉ mục giúp một số truy vấn tìm/lọc/sắp xếp nhưng tăng chi phí ghi và dung lượng. Chỉ mục ghép cần xem thứ tự cột phù hợp với điều kiện truy vấn. Dùng EXPLAIN trước khi thêm chỉ mục và đo trên dữ liệu có kích thước phù hợp; bảng rất nhỏ không chứng minh được lợi ích rõ ràng.

Câu hỏi ôn tập và bài tập vận dụng

1. Dễ: phân biệt COUNT(*) và COUNT(p.id) trong LEFT JOIN. Khóa chưa có thanh toán vẫn có một dòng kết quả JOIN, nhưng p.id là NULL nên COUNT(p.id)=0.
2. Trung bình: thống kê khóa không có học viên. LEFT JOIN enrollments và GROUP BY khóa, HAVING COUNT(e.id)=0; không đặt điều kiện khiến bảng trái bị loại.
3. Khó: hai callback đồng thời xác nhận cùng đơn. Thiết kế giao dịch khóa đơn, kiểm tra trạng thái và UNIQUE cho ghi danh; callback sau thấy đơn hoàn tất thì trả kết quả nhất quán, không tạo lại.

Tiêu chí tự đánh giá và nguồn

Kiểm tra dữ liệu rỗng, khóa không có học viên, giao dịch cùng giá trị, callback lặp và lỗi giữa hai bước ghi. Giải thích được lý do COUNT bị nhân và chứng minh rollback không để đơn hoàn tất mà thiếu quyền học.

https://dev.mysql.com/doc/refman/8.4/en/join.html

https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

https://dev.mysql.com/doc/refman/8.4/en/explain.html

Dự án: Thiết kế CSDL website học trực tuyến — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Bài thực hành: thiết kế học viên, khóa, ghi danh, quiz và kết quả cho một lab. Viết truy vấn danh sách chưa ghi danh, số học viên mỗi khóa và điểm theo quy tắc đã chọn. Chưa nộp bài khác điểm 0; không biến thiếu dữ liệu thành kết luận trượt.

Tự luyện: tạo dữ liệu rỗng, trùng, khóa có hai học viên, một học viên làm nhiều lượt và một người chưa làm. **Hướng giải:** xác nhận quy tắc lấy điểm đầu/cuối/cao nhất/trung bình trước khi viết AVG. Thử transaction bằng rollback trong lab; kiểm tra cả bản ghi con. Đối chiếu tổng báo cáo với truy vấn độc lập để phát hiện nhân dòng do JOIN.

Nguồn học tiếp

https://dev.mysql.com/doc/refman/8.4/en/tutorial.html